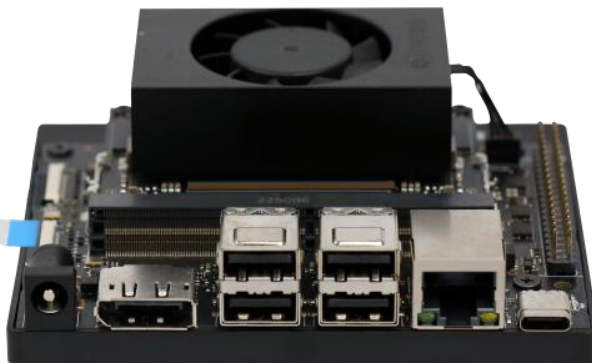


e-CAM121_CUONX

GStreamer Usage Guide



Version 1.2

e-con Systems

13/09/2023

Disclaimer

e-con Systems reserves the right to edit/modify this document without any prior intimation of whatsoever.

Contents

INTRODUCTION TO GSTREAMER.....	4
INSTALLATION PROCEDURE.....	5
TESTED GSTREAMER EXAMPLES.....	7
TROUBLESHOOTING.....	11
FAQ.....	12
WHAT'S NEXT?.....	13
GLOSSARY.....	14
SUPPORT.....	15

Introduction to GStreamer

GStreamer is a powerful and flexible multimedia framework with a lot of capabilities. It supports various features such as capturing, displaying, encoding, and decoding of image data. While using GStreamer, the possibilities of manipulating data are limitless.

The commands and output messages in this manual are represented by different colors as shown in the below table.

Table 1: Notation of Colors

Color	Notation
Blue	Commands running in development kit
Red	Output message in development PC
Orange	Commands running in client PC

This document explains how to install GStreamer-1.0 on the Jetson Orin Nano™/Orin™ NX development kit and use it with the available plugins.

Installation Procedure

This section describes how to install GStreamer-1.0 binaries and libraries.

To know the details of available plugins, please refer to the *Accelerated_GStreamer_User_Guide_Rev_<ver>.pdf* from [Jetson Download Center](#).

The steps to install GStreamer-1.0 binaries and libraries are as follows:

1. Run the following commands to install the GStreamer-1.0 binaries and libraries on the Jetson Orin Nano™/Orin™ NX development kit.

```
$ sudo apt-add-repository universe
$ sudo apt-get update
$ sudo apt-get install gstreamer1.0-tools gstreamer1.0-alsa
gstreamer1.0-plugins-base gstreamer1.0-plugins-good
gstreamer1.0-plugins-bad gstreamer1.0-plugins-ugly
gstreamer1.0-libav
```

Note: If the installation process stops with **could not get lock /var/lib/dpkg/lock** message, run the following command to remove the file and proceed with the installation.

```
$ sudo rm /var/lib/dpkg/lock
```

2. Run the following command to check the GStreamer-1.0 version.

```
$ gst-inspect-1.0 --version
```

Note: Make sure that e-CAM121_CUONX is connected, and the required drivers are loaded.

During booting, the module drivers for the e-CAM121_CUONX will be loaded automatically in the Jetson Orin Nano™/Orin™ NX development kit.

3. Run the following command to confirm whether the connected camera is initialized.

```
$ dmesg | grep "Detected eimx412 sensor"
```

The output message appears as shown below.

```
Detected eimx412 sensor
```

The output message indicates that camera connected is initialized properly.

4. Run the following command to check the presence of video node.

```
$ ls /dev/video*
```

The output message for camera setup appears as shown below.

```
/dev/video0
```

Tested GStreamer Examples

The section describes some of the tested GStreamer commands which work on the Jetson Orin Nano™/Orin™ NX development kit.

Example 1: Streaming 12 MP from a single camera resized to 1080p (HW accelerated)

Run the following command to stream 1080p video from a single camera.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> sensor-mode=<m> ! "video/x-raw(memory:NVMM), width=(int)4056, height=(int)3040, format=(string)NV12, framerate=<f>/1" ! queue ! nvvidconv ! "video/x-raw(memory:NVMM), width=(int)1920, height=(int)1080, format=(string)NV12" ! queue ! nv3dsink -e
```

Note: Replace <n> and <m> with the number of video device node and mode number from which you need to stream. Replace <f> with the maximum frame rate corresponding to the particular mode.

Example 2: Record 12 MP resized to 1080p in H.265 format to a video file (HW accelerated) in Orin™ NX development kit.

Run the following command to record 1080p video in H.265 format to a video file.

```
$ gst-launch-1.0 -v nvarguscamerasrc sensor-id=<n> sensor-mode=<m> ! "video/x-raw(memory:NVMM), width=(int)4056, height=3040, format=(string)NV12, framerate=<f>/1" ! nvvidconv ! "video/x-raw(memory:NVMM), format=(string)NV12, width=(int)1920, height=(int)1080" ! nvv4l2h265enc ! "video/x-h265, stream-format=(string)byte-stream" ! h265parse ! matroskamux ! queue ! filesink location=<filename>.mkv
```

Note: Replace <n> and <m> with the number of video device node and mode number from which you need to stream. Replace <f> with the maximum frame rate corresponding to the particular mode. Change <filename> with a name (For example, Video_1), to which the video will get stored.

Example 3: Playback of saved video file (HW accelerated)

Run the following command to playback the saved video.

```
$ gst-launch-1.0 filesrc location=<filename>.mkv ! matroskademux ! h265parse ! nvv4l2decoder ! nv3dsink
```

Note: Change <filename> with the name under which the video is recorded.

Example 4: Record 12 MP resized to 1080p in H.264 format to a video file in Orin™ NANO development kit.

```
$ gst-launch-1.0 -v nvarguscamerasrc sensor-id=<n> sensor-
mode=<m> ! "video/x-raw(memory:NVMM), width=(int)4056,
height=3040, format=(string)NV12, framerate=<f>/1" ! nvvidconv !
"video/x-raw(memory:NVMM), format=(string)NV12,
width=(int)1920, height=(int)1080" ! nvvidconv ! x264enc
bitrate=<bitrate> speed-preset=<preset> ! "video/x-h264,
stream-format=(string)byte-stream" ! h264parse ! matroskamux !
filesink location=<filename>.mkv
```

Note: Replace **<n>** and **<m>** with the number of video device node and mode number from which you need to stream. Replace **<f>** with the maximum frame rate corresponding to the particular mode. Replace **<bitrate>** and **<preset>** with you preferences, eg bitrate as 4000 and preset as superfast. Performance and quality of our encoding will depend on the value of bitrate and speed preset. Change **<filename>** with a name (For example, Video_1), to which the video will get stored.

Example 5: Playback of saved video file.

Run the following command to playback the saved video.

```
$ gst-launch-1.0 filesrc location=<filename>.mkv !
matroska-demux ! h264parse ! avdec_h264 ! autovideosink
```

Note: Change **<filename>** with the name under which the video is recorded.

Example 6: Network streaming H.264 encoded FHD video using RTP over UDP (HW accelerated) in Orin™ NX development kit.

Run the following command to stream the video captured by camera connected with the Jetson Orin™ NX development kit.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> sensor-
mode=<m> ! "video/x-raw(memory:NVMM), width=(int)4912,
height=3684, format=(string)NV12, framerate=<f>/1" !
nvvidconv ! "video/x-raw(memory:NVMM), format=(string)NV12,
width=(int)1920, height=(int)1080" ! nvv4l2h264enc ! rtph264pay
mtu=60000 ! udpsink clients=<ip_address>:<port_no>
sync=false
```

Replace **<f>** with the maximum frame rate corresponding to the particular mode. Replace **<n>** and **<m>** with the number of video device node and mode number from which you need to stream. Now **<ip_address>** with the IP address of the Client device (For example, 192.168.6.100) and **<port_no>** with the port number of the client device (For example, 5000).

Note: Do not close the application in the Jetson Orin™ NX development kit.

Run the following command in the client device to view the video.

```
$ gst-launch-1.0 udpsrc port=<port no.> caps="application/x-rtp, media=(string)video, encoding-name=(string)H264, sprop-parameter-sets=(string)"Z0JAKJWgHgCJ+VA\\=\\,aM48gA\\=\\=\\", payload=(int)96" ! rtph264depay ! decodebin ! videoconvert ! autovideosink sync=false
```

Note: Replace **<port_no>** with the port number of the client device (For example, 5000).

Example 7: Network streaming H.264 encoded FHD video using RTP over UDP in Orin™ NANO development kit.

Run the following command to stream the video captured by camera connected with the Jetson Orin™ NX development kit.

```
$ gst-launch-1.0 nvarguscamerasrc sensor-id=<n> sensor-mode=<m> ! "video/x-raw(memory:NVMM), width=(int)4912, height=3684, format=(string)NV12, framerate=<f>/1" ! nvvidconv ! "video/x-raw, format=(string)I420, width=(int)1920, height=(int)1080" ! x264enc bitrate=<bitrate> speed-preset=<preset> tune=zerolatency ! video/x-h264, stream-format=byte-stream ! h264parse ! rtph264pay mtu=60000 ! udpsink clients=<ip_address>:<port_no> sync=false
```

Replace **<f>** with the maximum frame rate corresponding to the particular mode. Replace **<n>** and **<m>** with the number of video device node and mode number from which you need to stream. Replace **<bitrate>** and **<preset>** with you preferences, eg bitrate as 4000 and preset as superfast. Now **<ip_address>** with the IP address of the Client device (For example, 192.168.6.100) and **<port_no>** with the port number of the client device (For example, 5000).

Note: Do not close the application in the Jetson Orin™ NX development kit.

Run the following command in the client device to view the video.

```
$ gst-launch-1.0 udpsrc port=<port no> ! application/x-rtp, payload=96 ! rtppjitterbuffer ! rtph264depay ! h264parse ! avdec_h264 ! autovideosink sync=false
```

Note: Replace **<port_no>** with the port number of the client device (For example, 5000).

Example 8: Capturing 12 MP still image

Run the following command to capture 12 MP still image.

```
$ gst-launch-1.0 nvarguscamerasrc tnr-mode=2 tnr-strength=1 ee-  
mode=0 sensor-id=<n> sensor-mode=<m> num-buffers=1 !  
"video/x-raw(memory:NVMM), format=(string)NV12,  
width=(int)4056, height=(int)3040, framerate=<f>/1" ! nvjpegenc !  
filesink location=<filename>.jpg
```

Note: Replace <n> and <m> with the number of video device node and mode number from which you need to stream. Change <filename> with a name (For example, Capture_1), to which the image will get stored. Replace <f> with the maximum frame rate corresponding to the particular mode.

Example 9: Streaming 480p cropped from 12mp

Run the following command to stream **480p cropped from 12mp**

```
$ gst-launch-1.0 nvarguscamerasrc tnr-mode=2 tnr-strength=1 ee-  
mode=0 sensor-id=<n> sensor-mode=<m> ! "video/x-  
raw(memory:NVMM), width=(int)4056, height=3040,  
format=(string)NV12, framerate=<f>/1" ! nvvidconv top=0  
bottom=480 left=3416 right=4056 ! "video/x-raw(memory:NVMM),  
width=(int)640, height=480, format=(string)NV12" ! nv3dsink -e
```

Note: Replace <n> and <m> with the number of video device node and mode number from which you need to stream. Replace <f> with the maximum frame rate corresponding to the particular mode.

Note: Make sure the cropped resolution is always fitting within the maximum width and height supported.

Troubleshooting

In this section, you can view the commonly occurring issue and their troubleshooting step.

During network streaming, I face issues with video quality and frame rate. How to overcome this issue?

Network streaming quality and frame rate depends on the network bandwidth and receiver decode capability. In the receiver end, if you use software decoder, the frame rate and quality will get affected. Therefore, avoid using software decoder in the receiver end to resolve this issue.

1. I have run the GStreamer command to stream camera as mentioned in this document, but I cannot get the maximum fps?

GStreamer uses the mmap method, which may cause the frame rate to drop. Additionally, there are restrictions in the GStreamer display plugin that will impact the frame rate.

In order to achieve the maximum frame rate, you can use the argus_camera application, by changing the exposure mode to manual mode and setting the appropriate value.

2. Why does the still capture sample GStreamer pipeline create improper auto white balance (AWB) and auto exposure (AE)?

The sample provided is just to demonstrate the still capture feature of the e-CAM121_CUONX. To get the desired result, run the following command to set the properties of nvarguscamerasrc plugin.

```
$ gst-inspect-1.0 nvarguscamerasrc
```

3. Why can't I encode the streaming using nvv4l2h265enc GStreamer command with Orin Nano™ development kit ?

The Jetson Orin Nano™ development kit does not include hardware units for video encoding (NVENC), unlike the other modules of the NVIDIA® Orin™ family.

What's Next?

After understanding the usage of GStreamer application, you can refer to the following documents to understand more about e-CAM121_CUONX.

- *e-CAM121_CUONX Datasheet*
- *e-CAM121_CUMI412C_MOD Datasheet*
- *e-CAM121_CUONX Lens Datasheet*
- *e-CAM121_CUONX Release Package Manifest*

Glossary

IP: Internet Protocol.

RTP: Real-time Transport Protocol.

UDP: User Datagram Protocol.

L4T: Linux for Tegra.

Contact Us

If you need any support on e-CAM121_CUONX product, please contact us using the Live Chat option available on our website - <https://www.e-consystems.com/>

Creating a Ticket

If you need to create a ticket for any type of issue, please visit the ticketing page on our website - <https://www.e-consystems.com/create-ticket.asp>

RMA

To know about our Return Material Authorization (RMA) policy, please visit the RMA Policy page on our website - <https://www.e-consystems.com/RMA-Policy.asp>

General Product Warranty Terms

To know about our General Product Warranty Terms, please visit the General Warranty Terms page on our website - <https://www.e-consystems.com/warranty.asp>

Revision History

Rev	Date	Description	Author
1.0	24-May-2023	Initial Draft	Product Marketing Team
1.1	01-July-2023	Support for both Orin Nano™ and Orin™ NX development kit.	Product Marketing Team
1.2	13-September-2023	Jetpack updation from 5.1.1 to 5.1.2 and added encoding examples for Orin Nano development kit.	Product Marketing Team